

[52] Kepler: Robust Learning for Parametric Query Optimization

요약

본 논문은 Doshi 등이 2023년 PACMMOD에 발표한 Kepler로, 학습 기반 쿼리 최적화를 위한 강건한 프레임워크를 제시한다. Kepler는 매개변수 쿼리(parametric query)의 최적화에 초점을 맞추며, 매개변수의 값이 변해도 일관되게 좋은 성능을 유지하는 실행 계획을 학습하는 것을 목표로 한다.

Kepler의 핵심 아이디어는 쿼리 매개변수가 다양한 값을 가질 때, 단순히 평균적인 최적 계획을 찾는 것이 아니라, 매개변수의 분포 변화에 강건한(robust) 계획을 찾는 것이다. 예를 들어, WHERE age > ?라는 쿼리에서 ?에 다양한 값이 들어올 때, 어떤 값에서는 A 계획이 최적이고 다른 값에서는 B 계획이 최적일 수 있다. Kepler는 이러한 매개변수 범위를 고려하여, 전반적으로 가장 좋은 성능을 제공하는 계획을 선택한다.

Kepler는 강건 최적화(robust optimization) 원리를 쿼리 최적화에 적용한다. 이는 최악의 경우(worst-case)에 대비하는 전략으로, 매개변수가 어떤 값을 가져도 합리적인 수준의 성능을 유지하도록 보장한다.

본 논문과 Exqutor의 관계를 살펴보면, 둘 다 학습 기반의 쿼리 최적화를 다루지만, 초점이 다르다. Exqutor는 주로 카디널리티 추정 정확도를 높이는 데 집중하는 반면, Kepler는 학습된 계획이 매개변수 변화에 얼마나 강건한지에 초점을 맞춘다. 두 접근법은 상호 보완적이다.

상세분석

52.1 매개변수 쿼리와 최적화의 도전

매개변수 쿼리(parametric query)는 일부 값이 런타임에 결정되는 쿼리이다. SQL에서 가장 흔한 예는 WHERE 절의 상수 값이 매개변수로 대체된 경우이다. 예를 들어: - 정적: SELECT * FROM orders WHERE price > 100 AND customer_type = 'premium' - 매개변수: SELECT * FROM orders WHERE price > ? AND customer_type = ?

매개변수 쿼리는 매우 흔하다. 대부분의 데이터베이스 애플리케이션에서 사용자 입력이나 런타임 조건에 따라 쿼리 조건이 변하기 때문이다. 또한 준비된 명령문(prepared statement)을 사용할 때도 매개변수 쿼리가 일반적이다.

매개변수 쿼리의 최적화는 여러 도전을 제시한다. 첫째, 매개변수 값에 따라 최적의 실행 계획이 완전히 달라질 수 있다는 점이다. 예를 들어: - price > 10000인 경우: 인덱스를 사용한 계획이 효율적 - price > 50인 경우: 풀 테이블 스캔이 더 효율적 (선택도가 높으므로)

둘째, 매개변수가 결정되기 전에 최적 계획을 선택해야 할 수도 있다는 점이다. 예를 들어, 컴파일 시간(compile time)에 계획을 선택해야 한다면, 매개변수의 구체적 값을 알 수 없다.

셋째, 매개변수 값이 예상과 다를 때 (예: 카디널리티 추정 오류), 선택된 계획의 성능이 심각하게 저하될 수 있다는 점이다.

전통적인 해결책은 일반화된 계획(generic plan)을 선택하는 것이다. 이는 대부분의 매개변수 값에 대해 합리적인 성능을 제공하는 계획이다. 하지만 이는 특정 매개변수 값에 대해 최적일 아닐 수 있다.

52.2 강건 최적화와 Kepler의 접근

Kepler는 강건 최적화(robust optimization) 원리를 도입한다. 강건 최적화의 기본 아이디어는 최악의 경우에 대비하는 것이다. 즉, 매개변수가 어떤 값을 가지더라도, 합리적인 수준의 성능을 보장하는 계획을 선택하는 것이다.

수학적으로, Kepler의 목표는 다음과 같이 표현할 수 있다:

최소화: $\max_{\text{parameter in } \Theta} \text{cost}(\text{plan}, \text{parameter})$

여기서 Θ 는 가능한 모든 매개변수 값의 범위이고, $\text{cost}(\text{plan}, \text{parameter})$ 는 특정 계획과 매개변수 조합에 대한 쿼리 실행 비용이다. 이는 최악의 경우 비용을 최소화하는 계획을 찾는 것을 의미한다.

이러한 접근은 직관적이지만 실무에서 구현하기 어렵다. 모든 가능한 매개변수 조합을 평가하는 것은 불가능하기 때문이다. Kepler는 다음과 같은 전략을 사용한다:

첫째, 매개변수 범위를 샘플링한다. 매개변수가 연속 범위인 경우, 균등하게 샘플링하거나, 특정 분포에 따라 샘플링한다.

둘째, 각 샘플에 대해 최적의 실행 계획을 결정한다. 이는 전통적인 쿼리 옵티마이저나 학습 기반의 방법을 사용할 수 있다.

셋째, 샘플된 모든 경우에서 좋은 성능을 제공하는 계획을 찾는다. 이는 다중 목표 최적화(multi-objective optimization) 문제로 볼 수 있다.

52.3 학습 기반의 계획 선택

Kepler는 머신러닝을 사용하여 매개변수로부터 실행 계획을 직접 예측한다. 입력은 매개변수 값(들)이고, 출력은 선택할 계획의 인덱스이다.

신경망 기반의 분류 모델을 사용하는 경우: - 입력층: 매개변수 값들 (정규화됨) - 숨은층: 매개변수와 계획 선택 간의 복잡한 관계를 학습 - 출력층: 각 가능한 계획에 대한 확률 (softmax 출력)

모델을 학습하기 위한 학습 데이터는 다음과 같이 수집된다: - 매개변수 샘플: 가능한 범위에서 여러 매개변수 조합을 생성 - 각 샘플에 대한 최적 계획: 전통적인 옵티마이저를 사용하거나, 모든 계획을 실행하여 가장 빠른 계획을 결정

학습 과정에서는 과적합을 방지하고, 강건성을 확보하기 위해 정규화와 데이터 증강(data augmentation)을 사용한다.

52.4 계획 안정성과 적응

Kepler는 단순히 각 매개변수 값에 대해 최적의 계획을 학습하는 것이 아니라, 계획의 안정성(stability)을 고려한다. 계획 안정성이란, 매개변수가 약간 변해도 동일한 계획을 선택하거나, 선택이 바뀌어도 성능 저하가 최소한이라는 의미이다.

이는 다음과 같은 이유로 중요하다: 1. 쿼리 플랜 캐싱: 동일한 계획을 캐싱하면, 옵티마이저 호출을 줄일 수 있다. 2. 성능 예측 가능성: 사용자는 안정적인 성능을 기대한다. 3. 계획 변경의 오버헤드: 계획이 자주 바뀌면, 캐시 효율성이 떨어진다.

Kepler는 계획 안정성을 향상시키기 위해 다음과 같은 기법을 사용한다: - 부드러운 경계(smooth boundaries): 두 계획 간의 전환이 급격하지 않도록 학습 - 히스테리시스(hysteresis): 계획을 바꾸기 전에 충분히 큰 성능 개선이 필요하도록 설정 - 신뢰도 임계값: 모델의 신뢰도가 낮을 때는 현재 계획 유지

52.5 실험과 성능 평가

Kepler의 실험은 여러 실제 데이터베이스 워크로드에서 수행된다. Join Order Benchmark (JOB)와 같은 표준 벤치마크도 사용된다.

성능 평가는 다음과 같은 지표를 사용한다: - 평균 쿼리 실행 시간: 모든 매개변수 조합에 대한 평균 - 최악의 경우 실행 시간: 가장 느린 매개변수 조합 - 계획 안정성: 계획 변경 빈도와 각 변경 시 성능 변화 - 학습 비용: 학습 데이터 수집과 모델 학습에 소요되는 시간

실험 결과에 따르면: - 평균적으로 전통적 옵티마이저보다 20~30% 더 빠름 - 최악의 경우 성능이 더욱 개선되어, 전통적 옵티마이저보다 50% 이상 빠를 수 있음 - 계획 안정성도 개선되어, 계획 변경 빈도가 50% 이상 감소

52.6 본 논문과의 관계

Exqutor는 카디널리티 추정에 초점을 맞추는 반면, Kepler는 학습된 계획의 강건성에 초점을 맞춘다. 하지만 두 작업은 공통점이 있다:

첫째, 둘 다 머신러닝을 사용하여 쿼리 최적화를 개선한다.

둘째, 둘 다 실무적 워크로드의 복잡성을 다룬다. Exqutor는 유사성 쿼리의 복잡한 카디널리티 추정을 다루고, Kepler는 매개변수 쿼리의 복잡한 최적화를 다룬다.

셋째, 둘 다 학습 모델의 강건성(robustness)을 고려한다. Exqutor는 분포 이동에 강건한 추정을 제공하려 하고, Kepler는 매개변수 변화에 강건한 계획 선택을 제공한다.

추가 제기 문제

1. **매개변수 범위의 정의:** 매개변수의 가능한 범위를 어떻게 정의할 것인가? 실제로는 매개변수가 제약 없이 변할 수 있지만, 학습은 제한된 범위에서만 수행된다.
2. **다중 매개변수:** 여러 매개변수가 결합되었을 때 (예: WHERE col1 > ? AND col2 < ?), 학습 데이터의 차원이 급격히 증가한다 (차원의 저주).
3. **새로운 데이터 패턴:** 데이터가 변해서 이전의 최적 계획이 더 이상 최적이지 아닐 때, 모델을 언제 어떻게 재학습할 것인가?
4. **해석가능성:** 모델이 특정 매개변수 값에 대해 특정 계획을 추천한 이유를 설명할 수 있는가?